DAC

giugno 2025

4 Access control

The following files are shown by an `ls -l` command on a typical Unix system:

```
-r-xr-s--x 1 charlie acct   512 08-07-2008 15:35 accounting
-r-------- 1 alice   acct   256 08-07-2008 15:23 accounts
-rwxr-xr-x 1 system  system 128 01-01-1970 00:01 editor
-rw-r--r-- 1 alice   users  768 08-07-2008 15:26 cv.txt
-r--r----- 1 bob     gurus  344 08-07-2008 15:25 exam
-r--r----- 1 alice   gurus  188 08-07-2008 15:26 solutions
```

Unix users `alice` and `bob` are both members of the group `users` only, while `charlie` is a member of the group `gurus` only. Application `editor` allows users to read and write files of arbitrary name, whereas application `accounting` only allows users to append data records to the file `accounts`. Draw up an access control matrix with subjects {`alice, bob, charlie`} and objects {`accounts, cv.txt, exam, solutions`} that shows for each combination of subject and object whether the subject is able to read (R), (over)write (W), or at least append (A) records to the respective object.

	accounts	cv.txt	exam	solutions
alice				
bob				
charlie				

▼ Soluzione

	accounts	cv.txt	exam	solutions
Alice	R, A	R, W		R
Bob	A	R	R	
Charlie	A	R	R	R

Spiegazione passo-passo:

1. Si parte considerando solo i permessi base dei proprietari e degli altri utenti. `Alice` può leggere il suo file accounts, leggere e scrivere sul suo file cv.txt, non può fare nulla su exam e può leggere il suo file solutions. `Bob` non può fare nulla su accounts, può leggere cv.txt come altro utente, può leggere il suo file exam e basta. `Charlie` non può fare nulla su accounts, può leggere cv.txt come altro utente e basta

	accounts	cv.txt	exam	solutions
Alice	R	R, W		R
Bob		R	R	
Charlie		R		

2. Si prosegue con i permessi base dei gruppi di appartenenza. Gli utenti del gruppo `users`, ossia `Alice` e `Bob`, possono solo leggere il file cv.txt, un permesso che hanno già. L'utente del gruppo `gurus`, ossia `Charlie`, può leggere i files exam e solutions.

	accounts	cv.txt	exam	solutions
Alice	R	R, W		R
Bob		R	R	
Charlie		R	R	R

3. Il file eseguibile editor dell'utente `System` non è un file di `root`, quindi, quando lanciato su altri files, può accedervi solo se, chi lo ha lanciato, ha i relativi permessi su quei files. Dato che `System` non ha altri files, gli altri utenti, quando lanciano questo eseguibile, lo possono fare solo sui files dove possono già scrivere e/o leggere, quindi la matrice degli accessi non cambia.
4. **Passaggio critico**: Il file eseguibile `accounting` si limita ad agire solo sul file `accounts`, ma `Charlie` come proprietario non può fare append in quanto non possiede `accounts` e non fa parte del gruppo `acct`, tuttavia c'è il SET GROUP ID abilitato con `s`, per cui quando `Alice`, `Bob` e `Charlie` andranno ad eseguire `accounting` come altri utenti, acquisiranno i permessi del gruppo `acct` il quale ha permesso di scrittura su `accounts` e quindi il file eseguibile `accounting` potrò fare append.

	accounts	cv.txt	exam	solutions
Alice	R, A	R, W		R
Bob	A	R	R	
Charlie	A	R	R	R

Nota: gli utenti non ottengono il permesso di scrittura `W` sul file `accounts`, lo acquisiscono momentaneamente mentre eseguono `accounting` per poter fare append `A` ma, terminata l'esecuzione, perdono il permesso, compresa `Alice` anche se è proprietaria di `accounts`. `Append` è un permesso di scrittura più debole di `Write` perchè consente solo di aggiungere in fondo ai file ma non di modificarli o cancellarli, se gli utenti o anche solo alice avessero il permesso di scrittura allora accounting potrebbe non servire e si andrebbe contro la specifica dell'esercizio e la gestione degli accessi in Unix dove i permessi permanenti sono esplicitati.

Matrice

	accounts	cv.txt	exam	solutions
Alice	R, A	R, W		R
Bob	A	R	R	
Charlie	A	R	R	R

- Permessi base da `ls -l`
- Permessi dati da `editor`
- Permessi dati da `users`
- Permessi dati da `gurus`
- Permessi dati da `accounting`

Legenda:

- R: permesso di lettura
- W: permesso di scrittura completa
- A: permesso di append (aggiungere contenuto alla fine)

luglio 2022

4. **Access Control**

This is a simplified dump for the `ls -l` shell command in the current folder.

```
                    Gr  others
-rw-rw-r-- alice    alice   1
-rw-rw-rw- bob      bob     2
-rw-r--r-- charlie  charlie 3
-rw-r--r-- bob      bob     4
-r-xr-xr-- alice    alice   append
-r-sr-xr-- bob      appnd   append-super
---x--x--x charlie  charlie editor
```

Unix users are **alice**, **bob** and **charlie**.

The `id` command for each user returns:

- `id alice`: uid=1000(alice) gid=1000(alice) groups=1003(appnd),1000(alice)
- `id bob`: uid=1001(bob) gid=1001(bob) groups=1001(bob)
- `id charlie`: uid=1002(charlie) gid=1002(charlie) groups=1002(charlie)

There are 3 executable files:

- **editor** lets you open a file with **R**ead and **W**rite permissions;
- **append**, as the name suggests, lets you **A**ppend a line to a given file;
- **append-super** does the same as **append**.

Draw up an access control matrix with subjects {alice, bob, charlie} and objects {1, 2, 3, 4} that shows for each combination of subject and object whether the subject will be able to read (**R**), (over)write (**W**), or at least append records (**A**) to the respective object.

▼ Soluzione

	1	2	3	4
Alice	R, W, A	R, W, A	R	R, A
Bob	R	R, W, A	R	R, W, A
Charlie	R	R, W	R, W	R

Spiegazione passo-passo:

1. Si parte dall'inizio, ossia i permessi basici. Per ogni file, procedendo una riga alla volta, si guarda i permessi che i rispettivi utenti hanno sui rispettivi file come proprietari o come "altri utenti". Non si guardano, per ora, i permessi dei gruppi.

	1	2	3	4
Alice	R, W	R, W	R	R
Bob	R	R, W	R	R, W
Charlie	R	R, W	R, W	R

2. Ora si inizia a guardare gli eseguibili, partendo da `append` che concede il permesso A (append) al suo proprietario. Attenzione che, quando un utente prova a lanciare un eseguibile, prima bisogna verificare se ha il diritto di esecuzione `X` su di esso (se è il proprietario è sempre vero salvo diversamente indicato) e poi se ha il permesso di modifica su file su cui l'eseguibile andrà poi ad agire, sono due permessi diversi. Permesso di esecuzione e permesso di accesso. L'eseguibile append può essere lanciato solamente da `alice` dato che nessun'altro utente fa parte del suo gruppo e non è previsto il permesso `X` per tutti gli altri utenti. Quando `alice` lancia `append` lo fa con i suoi permessi, ergo potrà modificare solo i file di sua proprietà. Si noti anche la sottile differenza tra Write e Append, il primo permesso è più forte perchè consente anche la cancellazione dei contenuti nel file, il secondo si limita alle sole aggiunte.

	1	2	3	4
Alice	R, W, A	R, W	R	R
Bob	R	R, W	R	R, W
Charlie	R	R, W	R, W	R

4. Ora si passa al secondo eseguibile, ossia `append-super` di proprietà di `bob`. Si parte facendo la stessa considerazione fatta per `append`, quindi `bob` avrà il permesso A (append) sui file di sua proprietà.

	1	2	3	4
Alice	R, W, A	R, W	R	R
Bob	R	R, W, A	R	R, W, A
Charlie	R	R, W	R, W	R

5. **Passaggio critico**, l'eseguibile `append-super` può essere lanciato dagli utenti che fanno parte del gruppo `appnd`, ossia da `alice`. Tale eseguibile ha il set-userID abilitato con il permesso `S`, questo significa che gli utenti appartenenti al gruppo `appnd` potranno lanciare l'eseguibile come se fossero `bob`, ossia con i permessi di `bob`. Agendo come se fossero `bob`, di conseguenza, potranno fare append sui file di `bob`.

	1	2	3	4
Alice	R, W, A	R, W, A	R	R, A
Bob	R	R, W, A	R	R, W, A
Charlie	R	R, W	R, W	R

6. L'ultimo eseguibile, ossia `editor`, è una supercazzola dato che consente a `charlie` di leggere e scrivere solo sui suoi file essendone proprietario, permessi che ha già, e consente a tutti gli altri utenti di lanciarlo ma con i loro permessi. Non essendo il set-userID abilitato nella parte dei permessi del proprietario, quando `alice` o `bob` lancieranno `editor` per ottenere i permessi di lettura e scrittura sui file di `charlie`, l'esecuzione fallirà.

Matrice

	File 1	File 2	File 3	File 4
alice	R, W, A	R, W, A	R	R, A
bob	R	R, W, A	R	R, W, A
charlie	R	R, W	R, W	R

- Permessi base da `ls -l`
- Permessi dati da `editor`
- Permessi dati da `append`
- Permessi dati da `append-super`

Legenda:

- R: permesso di lettura
- W: permesso di scrittura completa
- A: permesso di append (aggiungere contenuto alla fine)

gennaio 2022

4. Access Control

A restaurant processes orders with a FIFO strategy. Orders are stored in file `orders.txt`. Program `enqueue` modifies `orders.txt` by adding an order to the tail of the queue. Program `dequeue` modifies `orders.txt` by deleting the order at the head of the queue.

Define a UNIX-based ACL of the files so to ensure that the following access control policy is enforced by the operating system:

- `willie` and `winona`, the waiters, can add new orders, but cannot read nor delete existing orders;
- `charlie` and `cathy`, the cooks, can read and delete existing orders, but cannot add new orders nor carry out any other modification to `orders.txt`;
- `mario`, the maitre, must be able to read the orders (and possibly do other actions).

Please provide your answer by completing the dump of the `ls -l` shell command (assuming that all relevant files are in the current folder):

```
__________ _______ _______ orders.txt
__________ _______ waiters enqueue
__________ _______ cooks   dequeue
```

where `cooks` is a group containing `charlie` and `cathy` and `waiters` is a group containing `willie` and `winona`.

▼ Soluzione

```
- --- --- --- owner groupname filename
- rw_ r__ ___ mario cooks   orders.txt
- rws __x ___ mario waiters enqueue
- rws __x ___ mario cooks   dequeue
```

Spiegazione passo-passo:

1. Si parte dal verificare chi sono i proprietari dei file, in questo caso si parte da orders.txt e si procede per esclusione. I waiters, ossia willie e winona, non possono essere proprietari del file per due ragioni, la prima è che, in UNIX, un gruppo non può essere proprietario di un file e la seconda è che hanno delle restrizioni su cosa possono fare su quel file. Stesso ragionamento vale per i cooks, ossia charlie e cathy. Andando per esclusione l'unico utente che rimane è mario, il maitre.

```
- ___ ___ ___ mario _______ orders.txt
- ___ ___ ___ _____ waiters enqueue
- ___ ___ ___ _____ cooks   dequeue
```

2. Gli utenti che lanceranno gli eseguibili enqueue e dequeue dovranno poter modificare orders.txt secondo le direttive, ma lo potranno fare solo se avranno i permessi del proprietario di orders.txt, ossia mario, quindi egli dovrà essere il proprietario degli eseguibili.

```
- ___ ___ ___ mario _______ orders.txt
- ___ ___ ___ mario waiters enqueue
- ___ ___ ___ mario cooks   dequeue
```

3. Ora si passa a definire i permessi, partendo da quelli del proprietario di un file che sono i più semplici. Di base un proprietario ha tutti i permessi sui suoi files, ma attenzione che i file non eseguibili, come quelli di testo .txt, non possono essere eseguiti e il permesso di esecuzione su di essi non ha senso.

```
- rw_ ___ ___ mario _______ orders.txt
- rwx ___ ___ mario waiters enqueue
- rwx ___ ___ mario cooks   dequeue
```

4. Ora si passa ai permessi dei gruppi, i waiters devono poter lanciare la enqueue e i cooks la dequeue.

```
- rw_ ___ ___ mario _______ orders.txt
- rwx __x ___ mario waiters enqueue
- rwx __x ___ mario cooks   dequeue
```

5. Non sono previsti permessi da dare a qualsiasi utente esterno ai gruppi e diverso dal proprietario. A questo punto c'è da fare una osservazione non banale, i gruppi possono adesso sì lanciare gli eseguibili, ma con i loro permessi, ossia da NON proprietari del file orders.txt, ergo quando andranno ad aprire e modificare orders.txt gli eseguibili falliranno in quanto l'unico che può modificare tale file è mario. Serve il set-userid abilitato sui due eseguibili in modo che, quando vengono lanciati, essi vengano eseguiti con i permessi del loro proprietario, ossia mario.

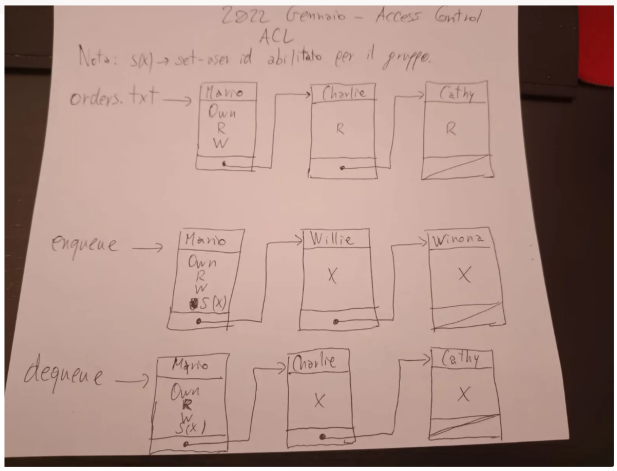
```
- rw_ ___ ___ mario _______ orders.txt
- rws __x ___ mario waiters enqueue
- rws __x ___ mario cooks   dequeue
```

6. Ora c'è da definire il gruppo del file orders.txt che in un primo momento potrebbe sembrare mario, infatti quando un file viene creato proprietario e gruppo coincidono.

```
- rw_ ___ ___ mario mario   orders.txt
- rws __x ___ mario waiters enqueue
- rws __x ___ mario cooks   dequeue
```

7. Rileggendo la specifica dell'esercizio si nota che i cooks devono poter leggere il file orders.txt, e non potendolo fare lanciando la dequeue lo devono poter fare come gruppo associato al file, avendo inoltre il permesso di poterlo leggere.

```
- rw_ r__ ___ mario cooks   orders.txt
- rws __x ___ mario waiters enqueue
- rws __x ___ mario cooks   dequeue
```



Approndimento extra:

Senza il bit SUID, l'intero sistema di gestione ordini diventerebbe inutilizzabile perché:

- I programmi sarebbero eseguibili ma inefficaci
- Nessuno tranne mario potrebbe modificare il file `orders.txt`
- Il requisito di sicurezza verrebbe rispettato, ma la funzionalità del sistema sarebbe compromessa

Il bit SUID è quindi essenziale per permettere l'esecuzione di operazioni privilegiate attraverso programmi specifici, senza dare agli utenti permessi diretti sul file dati.

In pratica, il bit SUID (s) fa sì che il programma venga eseguito con i privilegi di mario.

	orders.txt	enqueue	dequeue
mario	R, W, E, D	R, W, X	R, W, X
waiters	E	X	
cooks	D		X

Nota:

I waiters eseguono enqueue, prendono provvisoriamente i privilegi di Mario (proprietario), ma NON sono da mettere nella tabella (perchè sono provvisori solo per fare una azione).

Se Mario non avesse il diritto di scrivere, allora l'execute di enqueue non andrebbe a buon fine perchè prenderebbero i suoi privilegi, ma non potrebbero scrivere.

Giugno 2009

Controllo degli Accessi

Si consideri un sistema con tre utenti: Alice, Bob e Charlie.

Alice possiede il file alice.bat, Bob può solo leggerlo e scriverlo, mentre Charlie può solo eseguirlo.

Charlie può solo leggere il file bob.bat, che è posseduto da Bob, mentre Alice lo può solo leggere e scrivere.

Charlie possiede il file charlie.bat; Alice lo può solo scrivere e Bob può solo eseguirlo.

Ogni file può essere letto, scritto ed eseguito dagli utenti che lo posseggono.

a. Si scriva la matrice di controllo degli accessi corrispondente a tale situazione.

▼ Soluzione

	alice.bat	bob.bat	charlie.bat
Alice	*rwx*	*rw*	*w*
Bob	*rw*	*rwx*	*x*
Charlie	*x*	*r*	*rwx*

b. Si scriva la matrice di controllo degli accessi che si ottiene se Charlie da ad Alice il permesso di leggere charlie.bat e Alice revoca a Bob il permesso di scrivere alice.bat.

▼ Soluzione

	alice.bat	bob.bat	charlie.bat
Alice	*rwx*	*rw*	*wr*
Bob	*r*	*rwx*	*x*
Charlie	*x*	*r*	*rwx*

2007

6. ***Controllo degli Accessi*** *Si consideri un sistema con tre utenti: Alice, Bob e Charlie. Alice possiede il file* `alice.bat`*, Bob può solo leggerlo e scriverlo, mentre Charlie può solo eseguirlo. Charlie può solo leggere il file* `bob.bat`*, che è posseduto da Bob, mentre Alice lo può solo leggere e scrivere. Charlie possiede il file* `charlie.bat`*; Alice lo può solo scrivere e Bob può solo eseguirlo. Ogni file può essere letto, scritto ed eseguito dagli utenti che lo posseggono.*

(a) Si scriva la matrice di controllo degli accessi corrispondente a tale situazione.

▼ Soluzione

	alice.bat	bob.bat	charlie.bat
Alice	OWN, R, W, X	R, W	W
Bob	R, W	OWN, R, W, X	X
Charlie	X	R	OWN, R, W, X

(b) Si scriva la matrice di controllo degli accessi che si ottiene se Charlie dà ad Alice il permesso di leggere `charlie.bat` *e Alice revoca a Bob il permesso di scrivere* `alice.bat`*.*

▼ Soluzione

	alice.bat	bob.bat	charlie.bat
Alice	OWN, R, W, X	R, W	W, R
Bob	R	OWN, R, W, X	X
Charlie	X	R	OWN, R, W, X

Luglio 2008

2008 Luglio - DAC (permessi sui file)

Si consideri un sistema con tre utenti: Alice, Bob e Charlie.

Alice possiede il file alice.bat che può essere letto sia da Bob che da Charlie.

Charlie può leggere e scrivere il file bob.bat, che e posseduto da Bob, ma Alice lo può solo leggere.

Solo Charlie può leggere il file charlie.bat che gli appartiene.

Ogni file può essere eseguito dagli utenti che lo posseggono.

a. Si scriva la matrice di controllo degli accessi corrispondente a tale situazione.

▼ Soluzione

	alice.bat	bob.bat	charlie.bat
Alice	*ox*	*r*	
Bob	*r*	*ox*	
Charlie	*r*	*rw*	*orx*

b. Si scriva la matrice di controllo degli accessi che si ottiene se Charlie da ad Alice il permesso di leggere charlie.bat e Alice revoca a Bob il permesso di leggere alice.bat.

▼ Soluzione

	alice.bat	bob.bat	charlie.bat
Alice	*ox*	*r*	*r*
Bob		*ox*	
Charlie	*r*	*rw*	*orx*

Bell-LaPadula

Riassunto Gemini

Certamente! Questi esercizi riguardano il modello di sicurezza **Bell-LaPadula (BLP)**. Questo modello è progettato per garantire la **confidenzialità** dei dati (ovvero: chi non deve sapere, non sa).

Per risolvere questi esercizi, devi confrontare l'etichetta dell'**Utente (Soggetto)** con l'etichetta della **Risorsa (Oggetto)** o viceversa, applicando due regole ferree.

Ecco la guida passo-passo per capire i permessi.

1. La Struttura dell'Etichetta

Ogni etichetta è formata da due parti: (Livello, {Insieme di Categorie}).
Esempio: (Secret, {red, blue})

1. Il **Livello di Sicurezza (Gerarchico)**: C'è un ordine preciso.
   - Top Secret (TS) > Secret (S) > Confidential (C) > Unclassified (U).
   - Nota: Chi sta sopra "domina" chi sta sotto.
2. **Le Categorie (Insieme)**: Sono "compartimenti" (es. {red, blue}). Non c'è un ordine gerarchico, ma si guardano l'inclusione degli insiemi (insiemistica).

2. Le Due Regole d'Oro (Memorizza queste!)

Per decidere se l'utente può leggere o scrivere, devi verificare la relazione di **Dominanza**.
Diciamo che l'etichetta A **domina** l'etichetta B se:

- Il livello di A $\geq$ livello di B.
- E l'insieme delle categorie di A **contiene tutti** gli elementi di B (A $\supseteq$ B).

REGOLA A: "No Read Up" (Lettura)

Un utente può **LEGGERE** un documento solo se la sua etichetta **domina** quella del documento.

- **Logica:** Non puoi leggere cose più segrete del tuo livello o che riguardano progetti (categorie) di cui non fai parte.
- **Formula:** Livello Utente $\geq$ Livello Oggetto **E** Categorie Utente $\supseteq$ Categorie Oggetto.

REGOLA B: "No Write Down" (Scrittura)

Un utente può **SCRIVERE** su un documento solo se l'etichetta del documento **domina** quella dell'utente.

- **Logica:** Non puoi scrivere informazioni segrete su un foglio meno protetto (sarebbe una fuga di notizie, o leak). Puoi però scrivere "verso l'alto" (aggiungere info in un contenitore più sicuro).
- **Formula:** Livello Oggetto $\geq$ Livello Utente **E** Categorie Oggetto $\supseteq$ Categorie Utente.

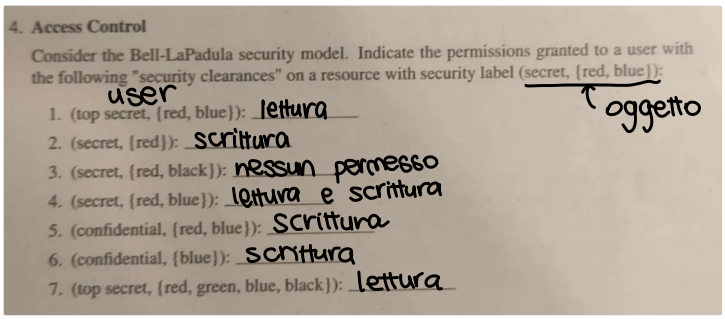
3. Come risolvere gli esercizi (Algoritmo mentale)

Quando leggi una riga dell'esercizio, fai questi tre controlli:

1. **Confronta i Livelli:** Chi è più alto?
2. **Confronta le Categorie:** Un insieme è contenuto interamente nell'altro?
3. **Applica i permessi:**

Relazione	Permesso	Spiegazione
Utente > Oggetto (in Livello e Categorie)	LETTURA	L'utente è superiore, può leggere i dati inferiori.
Utente < Oggetto (in Livello e Categorie)	SCRITTURA	L'utente è inferiore, può scrivere "verso l'alto" (blind write).
Utente == Oggetto (Identici)	LETTURA e SCRITTURA	Stesso livello, pieni poteri.
Incroci strani o Categorie mancanti	NESSUN PERMESSO	Se manca una categoria necessaria o i livelli non combaciano con la logica sopra.

Febbraio 2024



4. Access Control

Consider the Bell-LaPadula security model. Indicate the permissions granted to a user with the following "security clearances" on a resource with security label (secret, {red, blue}):

user ↑ oggetto

1. (top secret, {red, blue}): lettura
2. (secret, {red}): scrittura
3. (secret, {red, black}): nessun permesso
4. (secret, {red, blue}): lettura e scrittura
5. (confidential, {red, blue}): scrittura
6. (confidential, {blue}): scrittura
7. (top secret, {red, green, blue, black}): lettura

▼ Soluzione

1. (top secret, {red, blue}): LETTURA
2. (secret, {red}): SCRITTURA
3. (secret, {red, black}): NESSUN PERMESSO
4. (secret, {red, blue}): LETTURA e SCRITTURA
5. (confidential, {red, blue}): SCRITTURA
6. (confidential, {blue}): SCRITTURA
7. (top secret, {red, green, blue, black}): LETTURA

P.s. basta ricordarsi la legge suprema. Attenzione all'ultima etichetta perchè l'ordine delle classificazioni confonde ma {red, blue} ⊆ {red, blue, green, black} (!)

Luglio 2023

Consider the Bell-La Padula access control model and write the permissions granted to a user with the security label **(confidential, (red, blue))** over documents classified in the following way:

- (top secret, {red}): nessun permesso
- (secret, {red, blue}): scrittura
- (secret, {red, black}): nessun permesso
- (confidential, {white}): nessun permesso
- (confidential, {red, blue, green}): scrittura
- (confidential, {blue}): lettura
- (top secret, {red, green, blue, black}): scrittura

Gennaio 2019

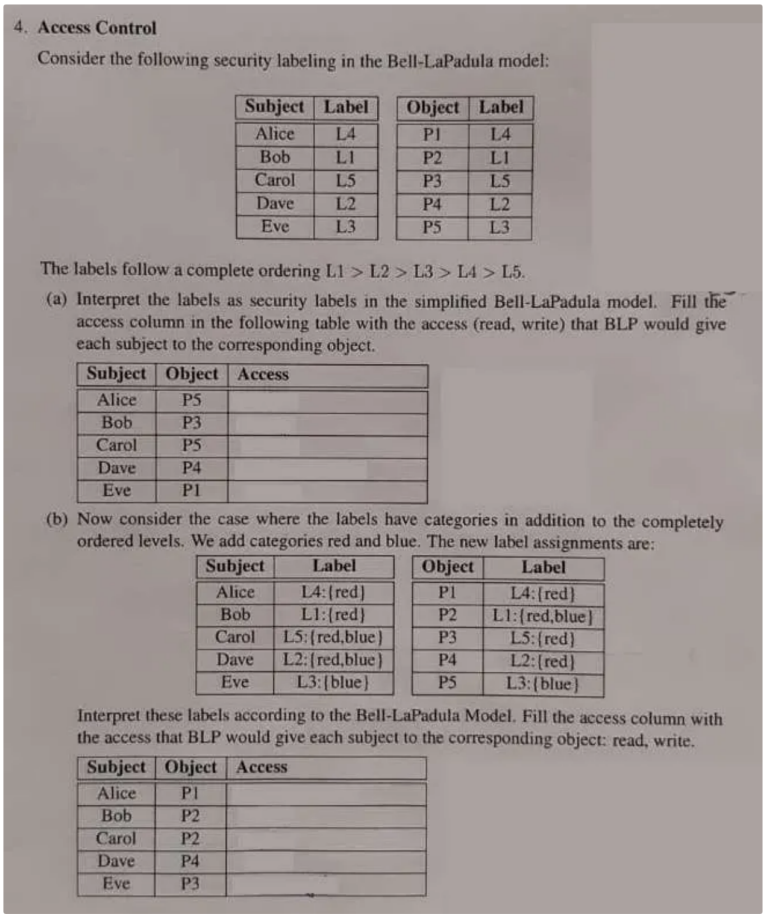
6. Controllo degli Accessi

Consider the Bell-La Padula access control model. Indicate the permissions granted to a user with *security clearance* (secret,{personnel, design, assistance}) on documents classified in the following way:

1. (top secret, {design}): nessun permesso
2. (top secret, {personnel, production, design, assistance}): scrittura
3. (secret, {personnel, assistance}): lettura
4. (secret, {production, design}): nessun permesso
5. (secret, {}): lettura
6. (confidential, {personnel, assistance}): lettura
7. (confidential, {production, design}): nessun permesso
8. (confidential, {}): lettura

Gennaio 2023

2023 Gennaio - Bell-LaPadula



4. **Access Control**

Consider the following security labeling in the Bell-LaPadula model:

Subject	Label
Alice	L4
Bob	L1
Carol	L5
Dave	L2
Eve	L3

Object	Label
P1	L4
P2	L1
P3	L5
P4	L2
P5	L3

The labels follow a complete ordering L1 > L2 > L3 > L4 > L5.

(a) Interpret the labels as security labels in the simplified Bell-LaPadula model. Fill the access column in the following table with the access (read, write) that BLP would give each subject to the corresponding object.

Subject	Object	Access
Alice	P5	
Bob	P3	
Carol	P5	
Dave	P4	
Eve	P1	

(b) Now consider the case where the labels have categories in addition to the completely ordered levels. We add categories red and blue. The new label assignments are:

Subject	Label
Alice	L4:{red}
Bob	L1:{red}
Carol	L5:{red,blue}
Dave	L2:{red,blue}
Eve	L3:{blue}

Object	Label
P1	L4:{red}
P2	L1:{red,blue}
P3	L5:{red}
P4	L2:{red}
P5	L3:{blue}

Interpret these labels according to the Bell-LaPadula Model. Fill the access column with the access that BLP would give each subject to the corresponding object: read, write.

Subject	Object	Access
Alice	P1	
Bob	P2	
Carol	P2	
Dave	P4	
Eve	P3	

Parte (a)

Soggetto	Oggetto	Accesso
Alice	P5	Scrittura
Bob	P3	Lettura
Carol	P5	Scrittura
Dave	P4	Lettura/Scrittura
Eve	P1	Lettura

Parte (b)

Soggetto	Oggetto	Accesso
Alice	P1	Lettura/Scrittura
Bob	P2	Scrittura
Carol	P2	Scrittura
Dave	P4	Lettura
Eve	P3	Nessun permesso

Giugno 2007

2007 Giugno - Bell-LaPadula

Si consideri il modello MAC di Bell-La Padula e si indichino i permessi concessi ad un utente con security label (secret, {personnel, design, assistance}) relativamente a documenti classificati nel seguente modo:

1. (top secret, {design}): nessun permesso
2. (top secret, {personnel, production, design, assistance}): lettura
3. (secret, {personnel, assistance}): lettura
4. (secret, {production, design}): nessun permesso
5. (secret, {}): lettura
6. (confidential, {personnel, assistance}): lettura
7. (confidential, {production, design}): nessun permesso
8. (confidential, {}): lettura

Altri esercizi

7. ***Access Control***

Consider the Bell-La Padula security model. Indicate the permissions granted to users with the following security clearances

1. *(top secret, {red, blue}):* lettura
2. *(secret, {red}):* scrittura
3. *(secret, {red, black}):* nessun permesso
4. *(secret, {red, blue}):* lettura e scrittura
5. *(confidential, {red, blue}):* scrittura
6. *(confidential, {blue}):* scrittura
7. *(top secret, {red, green, blue, black}):* lettura

over a resource with security label (secret, {red, blue}).

4. **Access Control**

Assume that Alice is granted security clearance

$$\langle Secret, \{dog, cat, pig\}\rangle$$

under the Bell-LaPadula security model. Write the permissions granted to Alice to object labeled as follows

(a) $\langle TopSecret; \{dog\}\rangle$ nessun permesso
(b) $\langle Secret; \{dog\}\rangle$ lettura
(c) $\langle Secret; \{dog, cow\}\rangle$ nessun permesso
(d) $\langle Secret; \{moose\}\rangle$ nessun permesso
(e) $\langle Confidential; \{dog, pig, cat\}\rangle$ lettura

ALTRO

2011

4. ***Controllo degli Accessi***

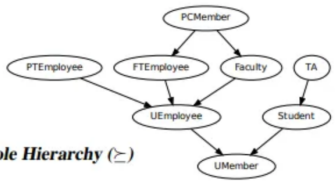
Si consideri la politica di controllo degli accessi RBAC presentata a lezione:

User Assignment (UA)

User	Role
Alice	PCMember
Bob	Faculty
Charlie	Faculty
David	TA
David	Student
Eve	UEmployee
Fred	Student
Greg	UMember

Permission Assignment (PA)

Role	Permission
PCMember	GrantTenure
Faculty	AssignGrades
TA	AssignHWScores
UEmployee	ReceiveHBenefits
Student	Register4Courses
UMember	UseGym



Role Hierarchy ($\succeq$)

e la politica ARBAC (sempre presentata a lezione):

- *can_assign1:* $UEmployee : \{Student, \neg TA\} \Longrightarrow PTEmployee$
- *can_assign2:* $UEmployee : \{UEmployee, \neg Faculty\} \Longrightarrow Student$
- *can_revoke:* $UEmployee : \{Faculty\} \Longrightarrow \neg Faculty$

a. E' possibile trasformare la politica in modo tale che David acquisisca il permesso Assign- `
Grades? Giustificare la risposta data.

▼ Risposta

No, non è possibile assegnare il ruolo Faculty a David.

b. E' possibile trasformare la politica in modo tale che Eve acquisisca il permesso Regi- ster4Courses? Giustificare la risposta data.

▼ Risposta

Si. E' possibile assegnare il ruolo Student ad Eve applicando can assign2.

c. E' possibile trasformare la politica in modo tale che Charlie acquisisca il permesso Regi- `
ster4Courses? Giustificare la risposta data.

▼ Risposta

Si, applicando can revoke e poi can assign2.

Giugno 2006

8. ***Controllo degli accessi***

Si consideri il seguente insieme di diritti {R,W,X,A,L,M,Own}*, dove* A*,* L *e* M *stanno per* Append, List *e* Modify.

(a) Utilizzando la sintassi del modello di Harrison-Ruzzo-Ullman, si scriva il comando per revoke_all_rights(s_1, s_2, o)*, con il quale* s_1 *cancella tutti i diritti di* s_2 *sulla risorsa* o*:*

▼ Soluzione

Soluzione.

command revoke_all_rights(s_1, s_2, o)
  delete R,W,X,A,L,M,Own **from** $M(s2, o)$
end

(b) Si indichi come deve essere modificato il comando affinché la cancellazione avvenga solo
se s_1 ha il diritto {Modify} su o.

▼ Soluzione

Soluzione.

command revoke_all_rights(s_1, s_2, o)
  if Modify $\in M(s1, o)$
  delete R,W,X,A,L,M,Own **from** $M(s2, o)$
end

SQL

```
### Versione più esplicita.
command revoke_all_rights(s_1, s_2, o)
    if M ∈ M(s_1, o)
    then
        delete R from M(s_2, o)
        delete W from M(s_2, o)
        delete X from M(s_2, o)
        delete A from M(s_2, o)
        delete L from M(s_2, o)
        delete M from M(s_2, o)
end
```

Luglio 2009

1. Controllo degli Accessi I
Si classifichino le seguenti situazioni come esempi di politiche di controllo degli accessi mandatory, discrezionali, o la loro combinazione. Si giustifichino le risposte date.

a. I meccanismi di controllo degli accessi del sistema operativo UNIX

▼ Risposta

Discrezionale.

b. Il caveau di una banca in cui solo i membri del consiglio di amministrazione possono entrare

▼ Risposta

Mandatory.

c. L'archivio di un ospedale comune dove la cartella clinica di un paziente P può essere
consultato per condurre analisi statistiche se P ha dato un permesso scritto che autorizzi
questa operazione.

▼ Risposta

Sia discrezionale che mandatory.

7. ***Controllo degli Accessi II***

Si consideri il seguente insieme di diritti {R,W,X,A,L,M,Own}*, dove* A*,* L *e* M *stanno per* Append, List *e* Modify.

(a) Utilizzando la sintassi del modello di Harrison-Ruzzo-Ullman, si scriva il comando per create(s, o)*, con il quale* s *viene creata la risorsa* o *e viene dato ad* s *il solo diritto di possesso* (Own)*.*

▶ Soluzione

(b) Utilizzando la sintassi del modello di Harrison-Ruzzo-Ullman, si scriva il comando per transfer.write(s_1, s_2, o)*, con il quale se* s_1 *ha il diritto di scrittura su* o *allora lo trasferisce (perdendolo) a* s_2*.*

Soluzione.
`command tranfer.write(s1,s2,o)`

▼ Soluzione

SQL

```
command transfert.write(s1,s2,o)
  if write ∈ M(s1,o)
  then
    enter W into M(s2,o)
    delete W from M(s1,o)
end
```

(c) Si indichi come deve essere modificato il comando affinché la cancellazione avvenga solo
se s_1 ha il diritto {Modify} su o e s_2 ha il diritto {Own} su o.

▼ Soluzione

Soluzione.

command revoke_all_rights(s_1, s_2, o)
  if Modify $\in M(s1, o)$ **and** Own $\in M(s2, o)$
  delete R,W,X,A,L,M,Own **from** $M(s2, o)$
end

SQL

```
### Versione più esplicita.
command revoke_all_rights(s_1, s_2, o)
    if M ∈ M(s_1, o) and O ∈ M(s_2, o)
    then
        delete R from M(s_2, o)
        delete W from M(s_2, o)
        delete X from M(s_2, o)
        delete A from M(s_2, o)
        delete L from M(s_2, o)
        delete M from M(s_2, o)
end
```

7. ***Controllo degli Accessi***

Si consideri il modello MAC di Bell-La Padula e si indichino i permessi concessi ad un utente con security label *(secret,* {*red, green, blue*}*) relativamente a documenti classificati nel seguente modo:*

1. *(top secret,* {*red*}*):* nessun permesso
2. *(secret,* {*red*}*):* lettura
3. *(secret,* {*red, black*}*):* nessun permesso
4. *(secret,* {*white*}*):* nessun permesso
5. *(confidential,* {*red, blue, green*}*):* lettura
6. *(confidential,* {*white*}*):* nessun permesso
7. *(top secret,* {*red, green, blue, black*}*):* scrittura

2006

Si consideri un sistema informativo universitario, in cui l'informazione relativa agli studenti è memorizzata in due files `piani_di_studio.xls` e `voti.xls.`

Siamo interessati a far sì che il sistema realizzi la seguente politica di sicurezza garantendo nel contempo che non vi possa essere alcun flusso informazione dal file `piani_di_studio.xls` al file `voti.xls` .

1. I `professori` possono `leggere` e `scrivere` i files `piani_di_studio.xls` e `voti.xls.`
2. Gli `esercitatori` possono `leggere` e `scrivere` il file `voti.xls` , ma non possono accedere in alcun modo al file `piani_di_studio.xls` .
3. Gli `studenti` possono solo `leggere` i files `piani_di_studio.xls` e `voti.xls` .

Si descriva un modello reticolare che consente di realizzare tale politica di sicurezza indicando gli utenti che sono autorizzati a fare downgrading all'atto del collegamento e quelli che invece non lo sono.

Si assuma che il sistema escluda le "write-up". Ovvero ogni tentativo di scrittura da parte di un processo p con security label s_p su una risorsa r con security label $s_r > s_p$ viene automaticamente negato dal sistema.

NOTA: Nel modello reticolare, normalmente un utente si collega al sistema utilizzando il proprio security label s. (Ciò significa che tutti i processi da lui invocati avranno security label s.)
Tuttavia, se autorizzato, un utente può fare downgrading, ovvero può collegarsi al sistema utilizzando un security label s′ ≤ s. (Ciò significa che tutti i processi da lui invocati avranno security label s′.)

professori (secret,{red,blue})

esercitatori(secret,{blue})

studenti(confidential,{red,blue})

piani_di_studio(confidential,{red})

voti(confidential,{blue})

▼ Soluzione

Attribuiamo ai files le security labels nel seguente modo:

- piani_di_studio.xls → (CONFIDENTIAL, {PIANI})
- voti.xls → (CONFIDENTIAL, {VOTI})

e attribuiamo agli utenti le security labels nel seguente modo:

- PROFESSORE → (SECRET, {PIANI, VOTI})
- ESERCITATORE → (SECRET, {VOTI})
- STUDENTE → (CONFIDENTIAL, {PIANI, VOTI})

Infine attribuiamo ai professori e agli esercitatori la possibilità di fare downgrading, mentre escludiamo tale possibilità per gli studenti.
Ora verifichiamo che tale modello realizza la politica di sicurezza richiesta:

1. I professori possono scrivere entrambe i files in quanto il loro security label domina
   quello dei files. Inoltre possono anche scrivere sui file purché facciano downgrading.
2. Gli esercitatori possono leggere e scrivere il file voti.xls (la lettura è possibile previo downgrading), ma non possono accedere in alcun modo al file piani_di_ studio.xls in quanto il proprio security label non domina quello di piani_di_ studio.xls (escludendo cosı la possibilita di lettura). `
3. Gli studenti possono leggere i files piani_di_studio.xls e voti.xls in quanto il loro security label domina quello dei files. Tuttavia non possono scrivere su tali files in quanto non possono fare downgrading.